OMNIX AI

AI Analytic Tower (AAT) – MVP Implementation Plan

Cursor-ready engineering blueprint | End-to-end workflow | Build sequence

Recommended stack: C# / ASP.NET Core .NET 10 LTS · Angular 20 · PostgreSQL · .NET Observer Worker · Ollama first integration

Purpose
Turn the AAT requirements into a practical, buildable first release. This plan prioritizes reliable AI observability before autonomous optimization and skill deployment.

Product principle
Observe → Analyze → Recommend → Approve → Deploy → Measure. The MVP implements the Observe and Analyze foundations first.

1. Executive recommendation

The uploaded PRD defines an enterprise control tower for on-premises AI/LLM monitoring, telemetry analytics, hardware correlation, AI optimization, approval, and secure pull-based skill updates. The recommended first release is a monitoring-first MVP: one organization, one observer agent, one Ollama integration, real telemetry ingestion, and a working dashboard.

Do not build the complete autonomous optimization pipeline before proving that telemetry is accurate, privacy-safe, reliable, and useful. Keep optimization and deployment behind later milestones.

2. Recommended technology stack

Layer	Technology	Why
Frontend	Angular 20 + TypeScript + Tailwind CSS	Admin dashboard, filters, charts, agent and execution screens
Backend	C# / ASP.NET Core Web API (.NET 10 LTS)	Enterprise APIs, auth, validation, background processing
Database	PostgreSQL + EF Core / Npgsql	Tenant data, agents, traces, metrics and audit records
Observer	.NET Worker Service	Long-running local agent, heartbeat, telemetry buffering and sync
LLM integration	Ollama first	Simple local runtime for the first end-to-end demo
API contract	REST + OpenAPI; batch ingestion	Straightforward integration and testing
Local infrastructure	Docker Compose	Repeatable PostgreSQL development environment
Later AI services	Python behind a provider/service interface	Use only where ML libraries materially help

Why not Python for everything?

Python has the strongest ML ecosystem, but the core product is an enterprise SaaS backend plus a long-running observer service. C# is a strong fit for both and aligns with the existing .NET/Angular development skill set. Keep Python optional for specialized model analysis later.

3. MVP scope

Included in the first monitoring release

- Tenant foundation: organization, environment, users and basic roles.
- Agent lifecycle: enrollment, machine authentication, heartbeat, version and online/offline status.
- LLM execution metadata: provider, model, request ID, token counts, TTFT, total latency, status and timestamps.
- Hardware samples: CPU, RAM, and GPU/VRAM where supported by the host.
- Reliable telemetry: bounded local buffer, asynchronous batch upload, retry/backoff and idempotent ingestion.
- Dashboard: overview KPIs, agent list, execution history, model analytics and hardware trends.
- Security: tenant isolation, secret handling, payload validation, structured logs and audit events.

Explicitly deferred

- Automatic prompt interception for every arbitrary application or framework.
- Raw prompt/completion capture or upload.
- AI-generated skill rewriting and external model analysis.
- Approval workflow, cryptographic artifact signing and hot deployment.

- Multi-region scale, Kubernetes, microservices and complex event streaming.

4. End-to-end architecture

Customer environment: AI application / Ollama → supported instrumentation → AAT Observer Worker → bounded local telemetry queue → outbound HTTPS batch upload.

AAT cloud: ASP.NET Core API → agent authentication and DTO validation → tenant mapping and idempotency → PostgreSQL → analytics queries → Angular dashboard.

The observer is not an inline proxy dependency. Telemetry transmission is best-effort and asynchronous. If the observer or cloud API fails, the LLM request must still complete.

5. End-to-end runtime flow

- Organization admin creates an organization and environment.
- Admin enrolls an observer agent using a one-time enrollment token.
- Observer authenticates and receives a stable agent identity/credential.
- Observer sends periodic heartbeat and version/health information.
- Customer enables a supported Ollama instrumentation path.
- Each LLM execution creates metadata-only telemetry with stable event and request IDs.
- Observer samples host CPU/RAM/GPU/VRAM with UTC timestamps.
- Observer adds events to a bounded local queue.
- Background sender batches events and uploads over outbound HTTPS.
- API authenticates the agent, validates limits, resolves tenant ownership and deduplicates event IDs.
- PostgreSQL stores execution and hardware records.
- Analytics APIs aggregate requests, tokens, latency, errors, model and hardware trends.
- Angular dashboard displays persisted, filterable data.

Failure path

If cloud connectivity fails, the observer buffers within a configured bound and retries with exponential backoff and jitter. When the buffer is full, apply a documented drop policy and increment a dropped-event counter. Never block inference while retrying.

6. Data privacy and correctness rules

- **Allowed:** event/request IDs, agent and application IDs, provider/model, token counts, timing, status, sanitized error code, timestamps and host metrics.
- **Prohibited:** raw prompts, completions, system instructions, API keys, authorization headers and arbitrary request/response bodies.
- Store all timestamps in UTC. Use stable event IDs for idempotent ingestion.
- Hardware samples are host-level. Request-window correlation is an estimate, not exact per-request GPU attribution unless the runtime provides trustworthy process-level measurements.
- Do not log secrets or sensitive payloads.

7. Core database entities

Entity	Key fields / purpose
Organizations	Id, Name, Status, CreatedAtUtc
Users / Roles	User identity, role assignment, tenant membership
Environments	Id, OrganizationId, Name, Type
Agents	Id, OrganizationId, EnvironmentId, Name, Version, Status, LastSeenUtc
AgentHeartbeats	AgentId, ReportedAtUtc, AgentVersion, HealthState
AIApplications	Id, OrganizationId, AgentId, Name, IntegrationType
Models	Provider, ModelName, Runtime
ExecutionTraces	EventId, RequestId, AgentId, ModelId, token counts, TTFT, latency, status, timestamps
HardwareMetrics	EventId, AgentId, sampled time, CPU, RAM, GPU, VRAM
AuditLogs	OrganizationId, UserId/AgentId, Action, Target, Timestamp

Add OrganizationId to tenant-owned data and enforce tenant filters in every query/command. Index common filters such as AgentId, Timestamp, ModelId and RequestId. Partition high-volume telemetry tables only when volume justifies it.

8. Initial API contract

Method	Endpoint	Purpose
POST	/api/v1/agents/enroll	Enroll agent
POST	/api/v1/agents/{agentId}/heartbeat	Agent heartbeat
GET	/api/v1/agents	List agents
POST	/api/v1/telemetry/executions/batch	Upload execution events
POST	/api/v1/telemetry/hardware/batch	Upload hardware samples
GET	/api/v1/executions	Paginated execution history
GET	/api/v1/analytics/overview	Dashboard KPIs
GET	/api/v1/analytics/models	Model analytics
GET	/api/v1/analytics/hardware	Hardware time series
GET	/api/v1/health	Service health

Agent endpoints require machine authentication. Dashboard endpoints require user authentication and tenant authorization. Enforce DTO validation, batch-size/payload limits, rate limits and pagination.

9. Suggested repository structure

```
OMNIX-AI/
■■■ frontend/omnix-dashboard/ # Angular 20
■■■ backend/
■ ■■■ AAT.API/
■ ■■■ AAT.Application/
■ ■■■ AAT.Domain/
■ ■■■ AAT.Infrastructure/
■ ■■■ AAT.Telemetry/
■ ■■■ AAT.Worker/
■■■ observer/AAT.Observer/ # .NET Worker Service
■■■ database/ # migrations / seed scripts
■■■ deployment/ # Docker and scripts
■■■ docs/ # architecture and flow
■■■ .cursor/
■■■ rules/omnix-ai-project.mdc
■■■ skills/omnix-ai-mvp/SKILL.md
```

Keep the first release as a modular monolith. Separate domain boundaries in code, but do not split into microservices prematurely.

10. Implementation roadmap

Phase	Scope	Exit criteria
0 – Foundation	Solution skeleton, Angular shell, Docker PostgreSQL, health endpoint, configuration	Backend and frontend build/run locally
1 – Identity & agents	Tenant model, login/roles, enrollment, machine auth, heartbeat, status	Agent can enroll and report heartbeat
2 – Telemetry API	Execution/hardware batch DTOs, validation, idempotency, storage	API stores safe, validated telemetry
3 – Observer	.NET worker, bounded queue, retry/backoff, Ollama integration	One Ollama workload produces metadata events

Phase	Scope	Exit criteria
4 – Dashboard	Overview, agents, execution history, model and hardware analytics	UI displays real persisted telemetry
5 – Hardening	Integration/load tests, tenant isolation, outage tests, security and deployment docs	Demo-ready monitored deployment
Later – Optimization	Analysis, skill proposals, approvals, signing, pull deployment, rollback	Only after monitoring MVP is stable

11. Definition of done / MVP acceptance

- A tenant admin can create/register an agent and view its status.
- Agent heartbeat updates last-seen time and online/offline status.
- One Ollama workload creates execution metadata events.
- No raw prompt or completion text is transmitted or stored.
- Agent sends telemetry asynchronously with bounded buffering and retry.
- API validates, authenticates, tenant-scopes and deduplicates events.
- Dashboard displays real persisted request, token, latency, error and hardware data.
- Execution history supports date/model/agent filters and pagination.
- Hardware charts show collection timestamps and source; GPU correlation is labelled estimated.
- Tests cover auth, tenant isolation, validation, idempotency and cloud outage behavior.

12. Cursor setup and first task

A Cursor starter pack accompanies this PDF. Copy its .cursor folder and docs folder into the root of a new OMNIX-AI repository. Cursor Agent can then use the reusable skill and project rules.

First prompt to paste into Cursor

```
Read `.cursor/rules/omnix-ai-project.mdc`, `.cursor/skills/omnix-ai-mvp/SKILL.md`, and
`docs/END_TO_END_FLOW.md`.

Implement Milestone 0 only: inspect the repository, create the .NET 10 solution and modular project
skeleton, create the Angular 20 frontend skeleton, add Docker Compose for PostgreSQL, add configuration
examples without secrets, and add a health endpoint. Do not implement telemetry, AI optimization,
approval, or deployment yet. Provide the file tree, exact setup commands, and run the build/tests. Do
not claim success unless commands are executed.
```

13. Local start checklist

- Install .NET 10 SDK, Node.js version supported by Angular 20, Angular CLI 20, Docker Desktop and Git.
- Create a new repository/folder named OMNIX-AI.
- Extract the Cursor starter ZIP into the repository root.
- Open the repository in Cursor and confirm the .cursor files are visible.
- Paste the first Cursor prompt above.
- Review generated solution structure and commands before accepting large changes.
- Run backend, frontend and database locally; verify /api/v1/health.
- Commit the clean foundation before moving to agent enrollment.

14. Important risks and design decisions

- The PRD describes a passive interceptor, but universal transparent interception is not automatically available for every LLM application. Begin with explicit instrumentation and supported integration adapters.
- The PRD's privacy promise and cloud-based prompt/skill optimization need a clear boundary. Keep skill content local by default; submit only explicitly approved/redacted content or use local analysis.
- GPU utilization is often host/device-level. Do not claim exact request-level resource usage from simple time overlap.

- Use agent enrollment, credential rotation/revocation, TLS, signed future artifacts, and least privilege.
- Do not implement Temporal or a distributed queue in the first milestone unless a real workflow durability requirement appears.

15. Product delivery sequence

First demo: one machine + Ollama + observer + API + PostgreSQL + Angular dashboard + real execution and hardware metrics.

Then: multi-agent/multi-environment support, alerts and reporting. Only after monitoring reliability is demonstrated, build the optimization, approval, signed artifact pull, staged rollout and rollback loop.

Prepared as an implementation blueprint based on the AAT PRD and the recommended monitoring-first MVP scope. Product decisions that are not explicit in the PRD are identified as recommendations in this plan.